

INTERIM REPORT - PNEUMONIA DETECTION

1. Summary of problem statement, data and findings.

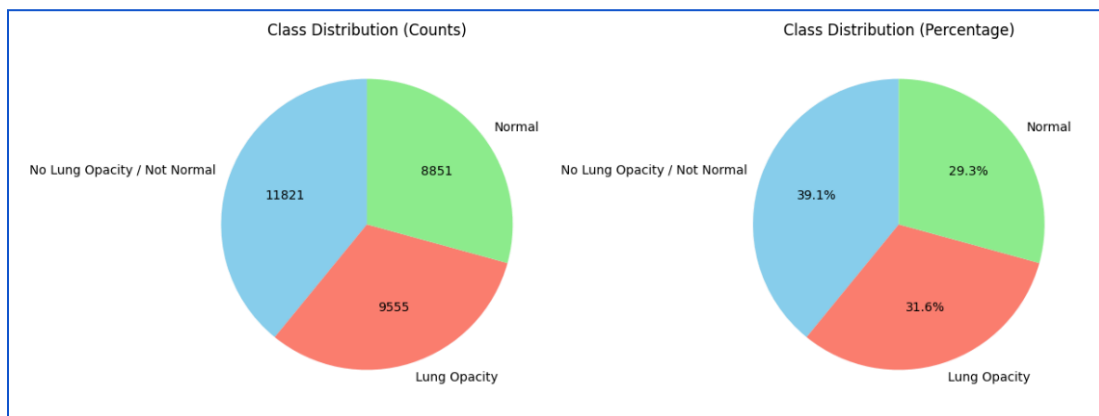
Problem Statements: Pneumonia is an inflammatory condition of the lungs that can be identified via chest radiographs. The project aims to develop a deep learning (DL) algorithm to automatically detect pneumonia by analyzing medical chest X-ray images.

Dataset: The dataset consists of DICOM images of chest radiographs provided in the RSNA Pneumonia Detection Challenge.

The data has three categories:

Categories	Class Distribution (Provided)
Normal	8,851 (~28.7%)
Lung Opacity (Pneumonia)	9,555 (~31.0%)
No Lung Opacity / Not Normal	11,821 (~38.3%)

* No Lung Opacity / Not Normal: abnormal but not pneumonia, sometimes mimicking pneumonia).



- The dataset shows mild class imbalance with **“No Lung Opacity / Not Normal”** being the **largest class**. The visual similarity between “Lung Opacity” and “Not Normal” cases increases the risk of misclassification and makes the task more challenging.

Findings (till Milestone 1):

- **Class Info Dataset** → [stage_2_detailed_class_info.csv](#)
 - **Shape:** 30,227 rows × 2 columns
 - Contains patient IDs and their associated **class labels** (e.g., Normal, Lung Opacity, No Lung Opacity / Not Normal).
- **Labels Info Dataset** → [stage_2_train_labels.csv](#)
 - **Shape:** 30,227 rows × 6 columns
 - Contains **bounding box annotations** for pneumonia cases along with patient IDs and target labels (1 = pneumonia present, 0 = no pneumonia).
- **Sample Submission Dataset** → [stage_2_sample_submission.csv](#)
 - **Shape:** 3,000 rows × 2 columns
 - Provides the required **submission format** for test predictions with patient IDs and predicted bounding boxes / probabilities.
- The training dataset is fully labeled with both **class information** and **bounding-box annotations**, supporting both classification and detection tasks, while the testing dataset contains only patient IDs and images with no labels.
- **Patient IDs** act as the key link between metadata (CSV files) and image files, ensuring correct mapping. Bounding-box annotations further provide localization details for accurate opacity detection.

2. Summary of the Approach to EDA and Pre-processing.

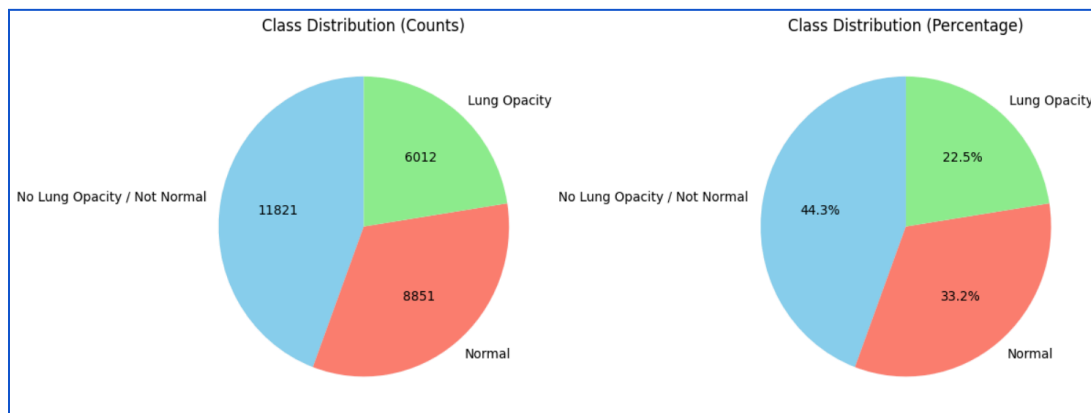
Exploratory Data Analysis (EDA):

- Class Distribution Analysis:
 - After removing duplicates, the dataset is imbalanced, with "No Lung Opacity / Not Normal" having nearly double the samples compared to "Lung Opacity."
 - This imbalance may bias the model, so mitigation techniques (class weighting, oversampling, or augmentation) will be considered.

Preprocessing Steps:

- **Remove Duplicates:** (No duplicates detected in "Normal" or "No Lung Opacity")

Categories	Class Distribution (Provided)	Class Distribution (after removing duplicates)
Normal	8,851 (~28.7%)	8,851 (~33.2%)
Lung Opacity (Pneumonia)	9,555 (~31.0%)	6,012 (~22.5%) ↓
No Lung Opacity / Not Normal	11,821 (~38.3%)	11,821 (~44.3%)



- **Image Availability:** Confirmed all patient IDs had corresponding DICOM images (no missing links).
- **DICOM → NumPy Conversion:** Converted images into arrays for normalization, resizing, and augmentation.
- **Bounding Box Normalization:** Rescaled pneumonia bounding boxes to match resized images (224×224).
- **Binary Target Distribution:** Pneumonia (22.5%) vs Non-pneumonia (77.5%) → confirms imbalance.
- **Train/Validation Split:** Data was split into 80/20 with stratification, preserving class proportions across both sets.
- **Visual Inspection:** Train images showed bounding boxes for pneumonia cases; test images lacked labels/bounding boxes (consistent with competition rules).

Data Integrity and Robustness:

- Verified train and test datasets against metadata.
- Skipped missing or unreadable images to maintain pipeline flow.
- Tracked dataset inconsistencies such as:
 - Missing patient IDs.
 - Filename mismatches between metadata and image files.
 - Corrupted or unreadable files.
 - Mixed formats (.dcm, .png, etc.).
 - Parsing issues due to uncommon compression/encoding.
- Ensured preprocessing pipeline remained robust against irregularities.

Implications

The findings from Milestone-1 highlight both the opportunities and challenges in developing a deep learning solution for pneumonia detection.

- The dataset supports both **classification** (pneumonia vs. non-pneumonia) and **localization** (bounding box for lung opacities), enabling development of interpretable models.
- **Mild class imbalance and visual similarity** between pneumonia and other abnormalities may lead to misclassification, requiring robust handling techniques.
- Automated detection systems can help **reduce radiologist workload** by acting as a triage tool and flagging high-risk cases.
- Such systems are particularly valuable in **resource-limited settings** where access to expert radiologists is limited.
- Faster and more accurate detection has the potential to **improve patient outcomes** through timely treatment and intervention.
- **Model interpretability** (e.g., bounding boxes or heatmaps) will be essential to build clinical trust and adoption.
- Ensuring **data diversity and external validation** is necessary to improve generalizability across different populations and imaging conditions.

3. Deciding Models and Model Building Based on the nature of the problem, decide what algorithms will be suitable and why?

Nature of the Problem:

- Since the problem is **image-based binary classification (Pneumonia vs. No Pneumonia)**, Convolutional Neural Networks (CNNs) are the most suitable.
- CNNs are well-suited because they can capture local patterns in chest X-rays (such as opacities or lesions) while maintaining translation invariance. They have shown strong performance in medical imaging tasks.
- CNNs can automatically learn hierarchical spatial features (edges → textures → regions → patterns like lung opacity), which traditional ML methods cannot capture effectively.
- **Basic CNN (custom architecture):** For the initial baseline (milestone 1), a simple CNN was built with three convolutional blocks followed by fully connected layers. This provided a benchmark performance and helped identify challenges such as overfitting.
- **Why not pre-trained models (at this stage of Milestone1):** Although pre-trained architectures like ResNet, VGG, or EfficientNet generally achieve higher accuracy, we first established a baseline with a custom CNN to better understand dataset characteristics, class imbalance, and preprocessing requirements.
- To Summarize, **a custom CNN** were first implemented as a baseline before exploring transfer learning with pretrained architectures (e.g., ResNet, DenseNet, EfficientNet) in the next phase.

Key Preprocessing Steps:

Data Integrity Checks:

- Verified that all patient IDs in the metadata had corresponding image files in the archives.
- Ensured there were no duplicate records in the metadata.

Missing Values:

- **No missing values** were found in the class labels.
- Missing values in bounding box coordinates corresponded to normal patients (Target = 0), where no pneumonia was present — this was expected and not an issue.

Image Preparation:

- Converted DICOM images into NumPy arrays.
- Normalized pixel intensity values to the range [0, 255] and resized images to 224 × 224 pixels.
- Stacked grayscale channels into 3-channel images for CNN compatibility.

Bounding Box Normalization:

- Bounding box coordinates were scaled to align with resized images for localization tasks.

Class Distribution:

- Dataset is imbalanced, with “No Lung Opacity / Not Normal” cases nearly double those of pneumonia-positive cases.
- This imbalance poses a risk of bias toward the majority class, requiring later use of augmentation or class balancing.

Dataset Splitting:

- Constructed a classification dataset with Target = 1 (pneumonia present) and Target = 0 (no pneumonia).
- Split into 80% training and 20% validation sets, using stratified sampling to preserve class distribution.

Model Design and Compilation:

- **Input Shape:** Images resized to **224×224×3** for computational efficiency and compatibility with standard CNN architectures.

- **Architecture:**
 - **Three convolutional + pooling blocks** for hierarchical feature extraction.
 - **Flatten** → **Dense(512)** → **Dropout(0.5)** → **Output** for classification.
 - Dropout introduced to reduce overfitting.
- **Output:** Single neuron with **sigmoid activation** for binary classification.
- **Compilation:**
 - Optimizer: **Adam (lr=0.0001)** for stable convergence.
 - Loss: **Binary Crossentropy**.
 - Metric: **Accuracy**.
- **Model size:** ~44M parameters → relatively large, but manageable given GPU support.

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 222, 222, 32)	896
max_pooling2d (MaxPooling2D)	(None, 111, 111, 32)	0
conv2d_1 (Conv2D)	(None, 109, 109, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 54, 54, 64)	0
conv2d_2 (Conv2D)	(None, 52, 52, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 26, 26, 128)	0
flatten (Flatten)	(None, 86528)	0
dense (Dense)	(None, 512)	44,302,848
dropout (Dropout)	(None, 512)	0
dense_1 (Dense)	(None, 1)	513

Total params: 44,396,609 (169.36 MB)

Trainable params: 44,396,609 (169.36 MB)

Non-trainable params: 0 (0.00 B)

Data Preparation:

- Implemented a **custom generator** to:
 - Load DICOM images on the fly (avoids memory overflow).
 - Resize and normalize images to `[0, 1]`.
 - Yield batches of images + labels.
- Ensured balanced batches across epochs via stratified sampling.
- Train/validation split: **80/20** stratified to preserve class proportions.

Training and Results:

Training Setup:

- **Batch Size:** 32
- **Epochs:** 10
- **Data Generators:** Custom data generator created to load DICOM images directly from zip archives, perform resizing and normalization, and yield batches of image-label pairs.
- **Optimizer:** Adam (learning rate = 0.0001)
- **Loss Function:** Binary Crossentropy
- **Metric:** Accuracy

Training Output:

```
Epoch 1/10
667/667 ————— 396s 581ms/step - accuracy: 0.7800 - loss: 0.4818 - val_accuracy: 0.7973 - val_loss: 0.4349
Epoch 2/10
667/667 ————— 317s 476ms/step - accuracy: 0.8119 - loss: 0.4223 - val_accuracy: 0.8016 - val_loss: 0.4186
Epoch 3/10
667/667 ————— 323s 485ms/step - accuracy: 0.8066 - loss: 0.4170 - val_accuracy: 0.8131 - val_loss: 0.4081
Epoch 4/10
667/667 ————— 318s 477ms/step - accuracy: 0.8139 - loss: 0.4109 - val_accuracy: 0.8061 - val_loss: 0.4285
Epoch 5/10
667/667 ————— 311s 468ms/step - accuracy: 0.8179 - loss: 0.3993 - val_accuracy: 0.8053 - val_loss: 0.4217
Epoch 6/10
667/667 ————— 318s 477ms/step - accuracy: 0.8271 - loss: 0.3802 - val_accuracy: 0.7989 - val_loss: 0.4269
Epoch 7/10
667/667 ————— 317s 476ms/step - accuracy: 0.8385 - loss: 0.3620 - val_accuracy: 0.8151 - val_loss: 0.4252
Epoch 8/10
667/667 ————— 316s 474ms/step - accuracy: 0.8510 - loss: 0.3312 - val_accuracy: 0.8080 - val_loss: 0.4351
Epoch 9/10
667/667 ————— 313s 469ms/step - accuracy: 0.8637 - loss: 0.3175 - val_accuracy: 0.8001 - val_loss: 0.4593
Epoch 10/10
667/667 ————— 317s 476ms/step - accuracy: 0.8815 - loss: 0.2803 - val_accuracy: 0.8037 - val_loss: 0.4726
166/166 ————— 59s 358ms/step - accuracy: 0.8078 - loss: 0.4717
Validation Loss: 0.4733
Validation Accuracy: 0.8063
```

Training Progress:

- **Epochs 1–2:**
 - Training accuracy improved from 78.0% → 81.1%, while validation accuracy increased slightly from 79.7% → 80.2%.
 - Both training and validation losses decreased, showing the model was effectively learning useful features.
- **Epochs 3–4:**
 - Training accuracy stabilized around 81.3%, and validation accuracy briefly peaked at 81.3% (Epoch 3).
 - Validation loss, however, began to fluctuate (dropping to 0.40 then rising again), indicating early signs of overfitting.
- **Epochs 5–7:**
 - Training accuracy rose steadily from 81.8% → 83.8%, while validation accuracy fluctuated between 80–81.5%.
 - Validation loss started to rise despite improvements in training accuracy, confirming divergence between training and validation performance.
- **Epochs 8–10:**
 - Training accuracy continued to climb sharply, reaching 88.1% by Epoch 10.
 - Validation accuracy dropped back to ~80.3%, showing no further generalization improvements.
 - Validation loss steadily increased from 0.42 → 0.47, confirming overfitting.
- **Final Evaluation:**
 - The model achieved Validation Accuracy = 80.6% and Validation Loss = 0.47.
 - These results suggest that while the CNN was able to learn strong discriminative features from chest X-rays, it overfit the training data after ~Epoch 4–5.

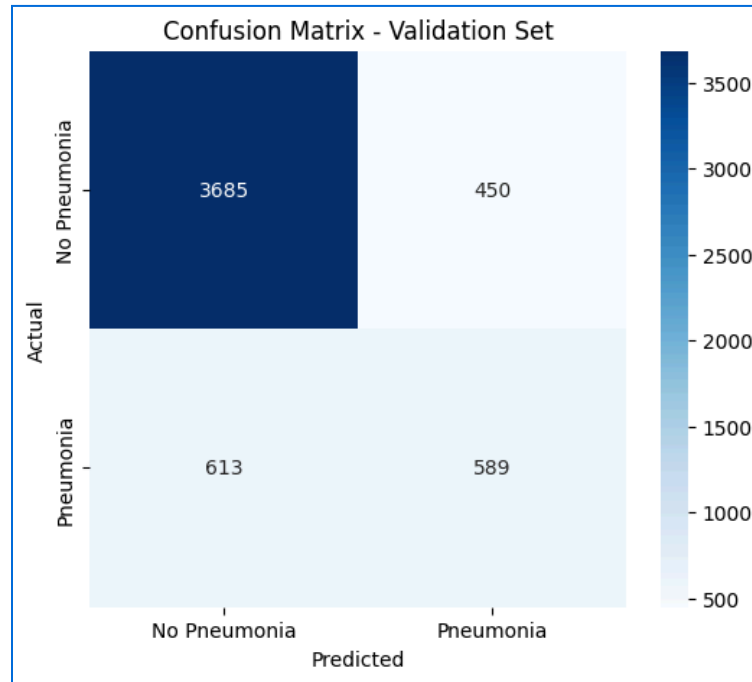
Training trend:

- Accuracy steadily increased from ~78% → ~88%.
- Validation accuracy plateaued around **80–81%**.

- Validation loss initially decreased but later diverged, suggesting **overfitting**.

Final performance (Validation Set):

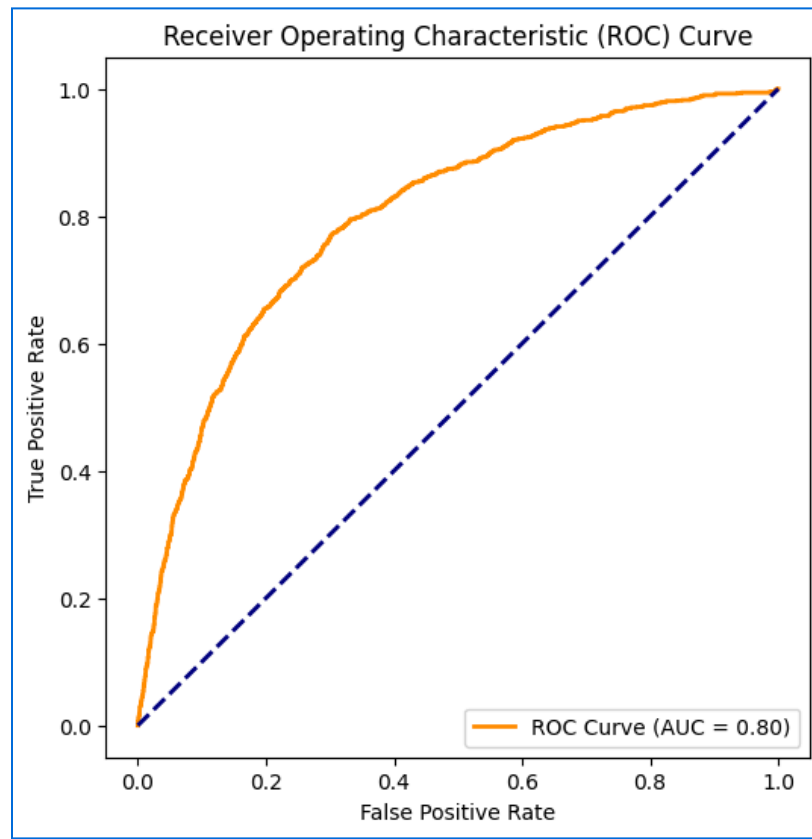
- Accuracy: ~**81%**
- Loss: ~**0.49**
- **Confusion Matrix (Validation Set):**



Classification Report:

	precision	recall	f1-score	support
No Pneumonia	0.86	0.89	0.87	4135
Pneumonia	0.57	0.49	0.53	1202
accuracy			0.80	5337
macro avg	0.71	0.69	0.70	5337
weighted avg	0.79	0.80	0.80	5337

ROC Curve:



- **Observations:**

- **Feature Learning:** The model appears to capture relevant features (such as lung opacities) early in training, which aligns with its strong ability to classify *No Pneumonia* cases.
- **Generalization Gap:** A sharp divergence between training and validation performance after Epoch 3 suggests overfitting — the model is memorizing training data instead of learning robust, generalizable patterns.
- **Model Complexity:** The presence of a very large fully connected layer (~44M parameters) increases overfitting risk, especially with an imbalanced dataset where pneumonia cases are fewer.
- **Performance Trade-off:** While the model achieves good overall accuracy (80%) and AUC (0.80), recall for pneumonia (0.49) is

significantly lower, showing that overfitting disproportionately impacts minority-class detection.

- **Clinical Implication:** High false negatives (missed pneumonia cases) are concerning in a healthcare setting, meaning the current model is safer for ruling out pneumonia than for confidently diagnosing it.

4. How to improve your model performance?

The baseline CNN achieved ~81% validation accuracy but showed signs of overfitting after a few epochs. Several strategies can be applied across data preparation, feature manipulation, and model design to improve performance:

Data-Level Improvements (Data Manipulation & Augmentation):

- **Data Augmentation:** Apply transformations such as random rotations, horizontal flips, zooming, shifting, and brightness adjustments to improve generalization and prevent the model from memorizing training samples.
- **Class Imbalance Handling:** Since “No Lung Opacity” cases outnumber pneumonia-positive cases, techniques like class weighting, oversampling minority classes, or synthetic augmentation (SMOTE for images) can balance the dataset.
- **Image Normalization/Standardization:** Standardizing pixel intensity ranges can stabilize training across batches.

Feature-Level Improvements (Representation Learning):

- **Bounding Box Utilization:** Instead of treating the task purely as classification, leverage bounding box annotations to train the model in a detection/localization setting (e.g., Faster R-CNN, YOLO). This focuses the model on relevant lung regions.
- **ROI Cropping:** Pre-crop lung regions based on bounding box coordinates before feeding to the CNN, reducing noise from non-lung areas.
- **Feature Extraction from Pretrained Models:** Use convolutional layers from pretrained architectures (ResNet, EfficientNet) as feature extractors and fine-tune them for pneumonia detection.

Model-Level Improvements:

- **Early Stopping:** Stop training once validation loss starts increasing to avoid overfitting.

- **Regularization:** Increase Dropout rate or add L2 weight decay in dense/convolutional layers to reduce model complexity.
- **Reduce Parameters:** Replace the Flatten → Dense(512) block (responsible for ~44M parameters) with Global Average Pooling (GAP). This drastically reduces parameter count and helps prevent overfitting.
- **Transfer Learning:** Fine-tune pretrained CNNs (ResNet, DenseNet, EfficientNet) that have shown strong performance on medical imaging tasks.
- **Advanced Architectures:** Experiment with specialized architectures like U-Net (for segmentation) or Faster R-CNN (for detection) to jointly improve classification and localization.

Conclusion:

The baseline CNN model demonstrated the ability to learn discriminative features from chest X-rays, achieving a validation accuracy of ~81%. However, training trends revealed signs of overfitting after a few epochs, highlighting the need for improved regularization, data augmentation, and transfer learning. The analysis so far establishes a strong foundation, and with the planned model improvements, performance and generalization are expected to significantly improve.

Flow Diagram / Chart

